

磐古前端说明文档

选用框架

vue-cli(vue3)+electron。

vue3用于前端框架，electron用于将前端内容打包成为一个桌面版的程序。

IDE: vscode or webstorm(我用的这个)

项目部署

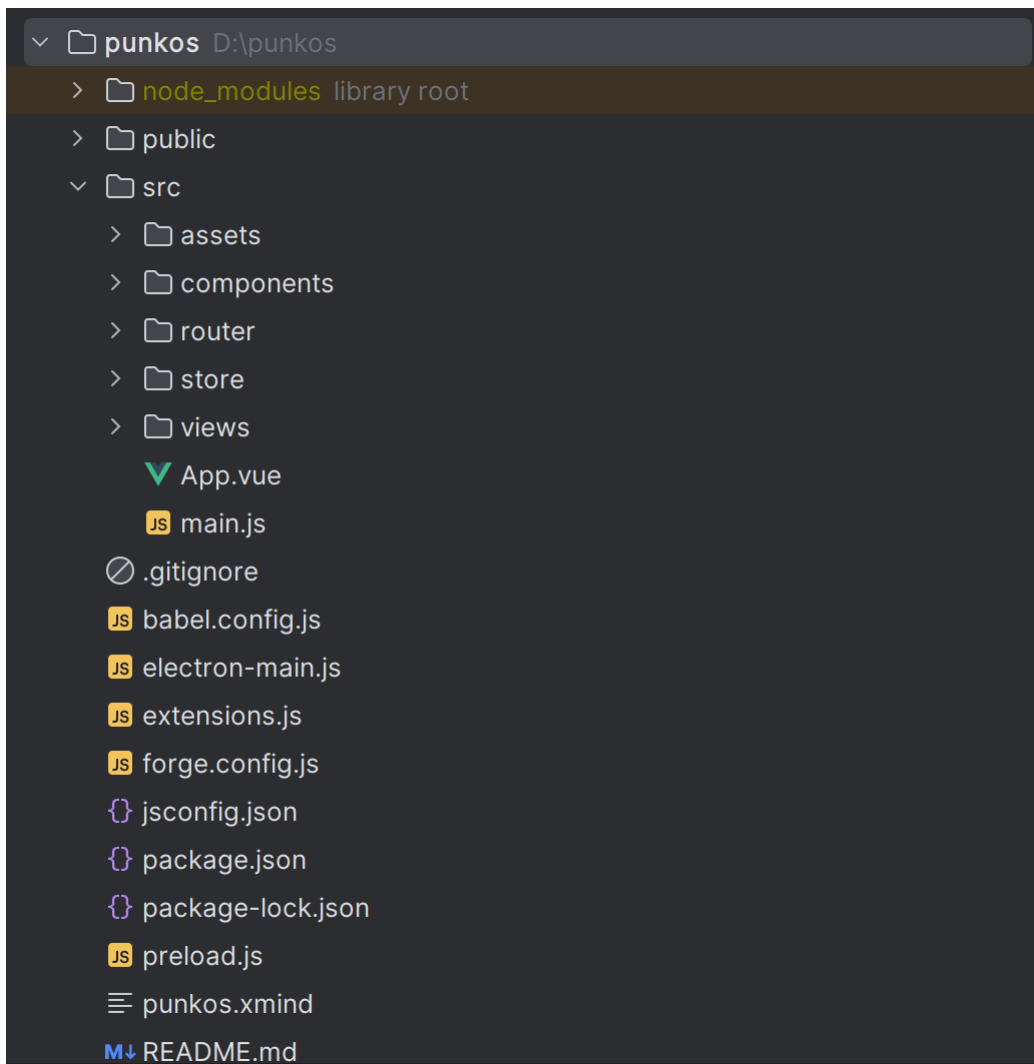
```
npm install    //下载相关依赖

npm serve     //启动vue项目

npm start     //启动electron项目并自动打包为一个桌面端可运行程序

// 更多指令参考package.json scripts{}中的内容
```

项目框架简单介绍



如图所示，目前项目文件结构如图，其中，node_modules文件夹主要存放依赖库，public暂时用不上，src是主要编写代码的地方。

- **src/asset** : 存放一些静态资源，如图片，全局使用的css等等。

- components
 - buttons
 - addnodeButton.vue
 - loginButton.vue
 - topnav
 - index.vue
 - HelloWorld.vue
 - MainBackground.vue
 - myPagination.vue
 - myWallet.vue
 - searchBar.vue

buttons: 封装了两个按钮组件，登录按钮的组件和addmodeButton的组件，后续使用button时，可以替换原生button这两个组件之一，用以改善button的视觉效果。

topnav: 顶部左侧的可伸缩导航栏

HelloWorld.vue: 新建项目时自动生成的，在项目中并没有使用。

MainBackground.vue: 定义了界面的背景，将界面按照左中右，分别切割，然后每一部分再按上中下进行div布局。在中间的部分使用slot进行插入，确保所编写的其余界面均在中间部分进行展示而不影响其他部分。

myPagination.vue: 自定义的一个分页器，此前使用场景背景为白色，还没做界面背景适配的工作。

myWallet: 钱包操作的主要内容，包括了通过ethers.js连接metamask，获取metamask地址和余额，钱包切换样式等等页面和功能。

searchBar: 搜索框的样式

- **src/router/index.js:** 存放路由，路由决定了页面的跳转，具体使用可以参考相关文档。简而言之，router提供了一种路由方式，组件页面之间跳转只需在router中进行配置即可。
- **src/store/index.js** 涉及的技术栈为vuex，以下简单贴一个vuex的介绍：

Vuex 是一个专为 Vue.js 应用程序开发的状态管理模式及其库。当在大型应用中处理多个组件共享的状态时，状态管理变得尤为重要。Vuex 提供了一个中央存储来管理这些共享的状态，并以特定的方式（遵循特定的规则）允许你改变状态。

下面是 Vuex 的主要概念和部分：

1. State

- 这是你的应用的单一数据源。
- 它是一个普通的 JavaScript 对象。

2. Getters

- 用来从 state 中派生出一些状态。
- 可以认为是 store 的计算属性。

3. Mutations

- 是更改状态的方法。
- 每个 mutation 都有一个字符串的事件类型和一个回调函数。回调函数是实际进行状态更改的地方，并接受 state 作为第一个参数。
- 必须是同步函数。

4. Actions

- 类似于 mutation，但它提交的是 mutation，而不是直接改变状态。
- 可以包含任意异步操作。

5. Modules

- 对于大型应用，可以将 store 分割成模块。
- 每个模块有自己的 state、mutations、actions、getters，甚至是嵌套子模块。

工作流程:

1. 组件内部通过 `dispatch` 方法触发 actions。
2. Actions 通过 `commit` 方法提交 mutations。
3. Mutations 会直接更改 state。
4. Vue 组件从 state 中获取数据，反映最新的状态。

为什么使用 Vuex?

- 当在一个大型单页面应用中，多个组件依赖于共享状态时，Vuex 可以帮助你组织和管理这些状态，确保其一致性。
- Vuex 也与 Vue 官方开发者工具集成，提供了时间旅行调试和状态快照导出等高级功能。

总之，Vuex 提供了一个结构化的、集中式的方式来管理 Vue.js 应用中的状态和逻辑，特别是在多个组件需要共享或响应相同的状态时。

- **src/views:** 主要编写代码的文件，用于书写页面级别的组件，区别于components(全局通用组件)。

本项目views下面目前文件夹即文件较多，每个区定义了一个入口vue文件，以及对应的文件夹。两种命名方式，一种为功能命名，而例如main-center一类的命名则是根据其在界面中的位置进行的命名。

如何写一个vue组件

在 Vue 组件中，通常有三个主要部分：`<template>`、`<script>` 和 `<style>`。这三部分共同构成了 Vue 组件的完整定义。

整体框架

- 这部分定义了组件的 HTML 结构或标记。
- 在这里，你可以使用 Vue 的模板语法，如指令 (`v-if`, `v-for`, `v-bind`, `v-on` 等) 和插值 (使用 `{{ }}` 对数据进行绑定)。
- `<template>` 部分最终被编译为一个渲染函数，这个函数在组件实例化时用于生成虚拟 DOM。
- 这部分定义了组件的逻辑。
- 你可以在这里定义组件的 `data`、`methods`、`computed` 属性、`watchers` 以及其他 Vue 组件选项。
- 也可以导入其他组件或外部模块，使其在当前组件中可用。
- 这部分定义了组件的样式。
- 你可以在这里写普通的 CSS，或者使用预处理器如 SCSS、LESS，只要你的项目配置支持这些预处理器。
- 使用 `scoped` 属性 (即 `<style scoped>`) 可以使样式仅应用于当前组件，防止样式泄露到其他组件。

简单来说，template用于书写html，以及绑定数据或者事件，script用于书写组件的具体逻辑和操作，例如点击事件函数等等。

css用于展示样式。

script

在 Vue 组件中，`<script>` 标签内定义了组件的逻辑。其中，`data`、`methods` 和生命周期钩子是最常用的部分。以下是它们的简要介绍：

1. data:

- **概念：**`data` 是 Vue 组件的响应式对象，用于存储组件的状态或数据。它应该是一个函数，该函数返回一个对象。
- **用法：**在组件模板或其他地方，你可以使用 `this` 来访问 `data` 中的属性。

```
data() {  
  return {  
    message: "Hello Vue!"  
  };  
}
```

2. methods:

- **概念：**`methods` 对象包含了该组件中的所有方法。这些方法可以被模板、生命周期钩子或其他方法调用。
- **用法：**你可以在模板中使用 `v-on` 指令或简写 `@` 来调用方法。

```
methods: {  
  sayHello() {  
    alert(this.message);  
  }  
}
```

在模板中使用：

```
<button @click="sayHello">Click me</button>
```

3. 生命周期钩子:

- **概念：**生命周期钩子是一系列在组件的不同生命周期阶段调用的特殊方法。这些方法允许你在特定时刻执行某些操作，例如在组件被创建、插入文档、更新或销毁时。
- **常见的生命周期钩子：**
 - `beforeCreate`：在组件实例刚被创建，但还未初始化属性和方法时调用。
 - `created`：在组件实例被创建并完成了数据属性的初始化后调用。
 - `beforeMount`：在模板被编译和挂载之前调用。
 - `mounted`：在模板被编译和挂载后调用。
 - `beforeUpdate`：在组件的数据发生变化，但尚未重新渲染 DOM 时调用。
 - `updated`：在组件的数据发生变化并重新渲染 DOM 后调用。
 - `beforeDestroy`：在组件实例被销毁之前调用。

- `destroyed`: 在组件实例被销毁后调用。
- **用法**: 只需在组件对象中定义相应的生命周期方法。

```
mounted() {  
  console.log("Component is mounted to the DOM!");  
}
```

通过合理地使用 `data`、`methods` 和生命周期钩子，你可以为 Vue 组件定义数据、功能和逻辑，从而构建丰富、互动的应用程序。

一些配置文件及常见指令介绍

1. `main.js`:

- 这是 Vue 项目的入口文件，通常用于创建 Vue 实例、挂载应用、引入外部插件或库等。
- 它与 Vue 应用的启动直接相关。

2. `electron-main.js`

- 这是 Electron 应用的主进程文件。
- 它负责创建和管理浏览器窗口，处理与系统级功能的交互，例如菜单、通知、文件系统操作等。
- Electron 应用的生命周期（如启动、关闭）通常在这里控制。

3. `preload.js`:

- Preload 脚本用于预加载一些操作，从而在 Electron 的 `Browserwindow` 中的网页上下文中提供这些操作。
- 由于 Electron 从主进程到渲染进程的通信推荐使用更安全的 IPC (Inter-Process Communication) 方法，`preload.js` 提供了一个桥接这两个进程的方法，使其能在网页上下文中使用，但不暴露全部 Node.js API。

4. `package.json`:

- Node.js 项目的配置文件，用于定义项目的元数据、依赖、脚本等。
- 在结合 Electron 的 Vue 项目中，这里通常会包括 Electron 的构建和启动脚本。

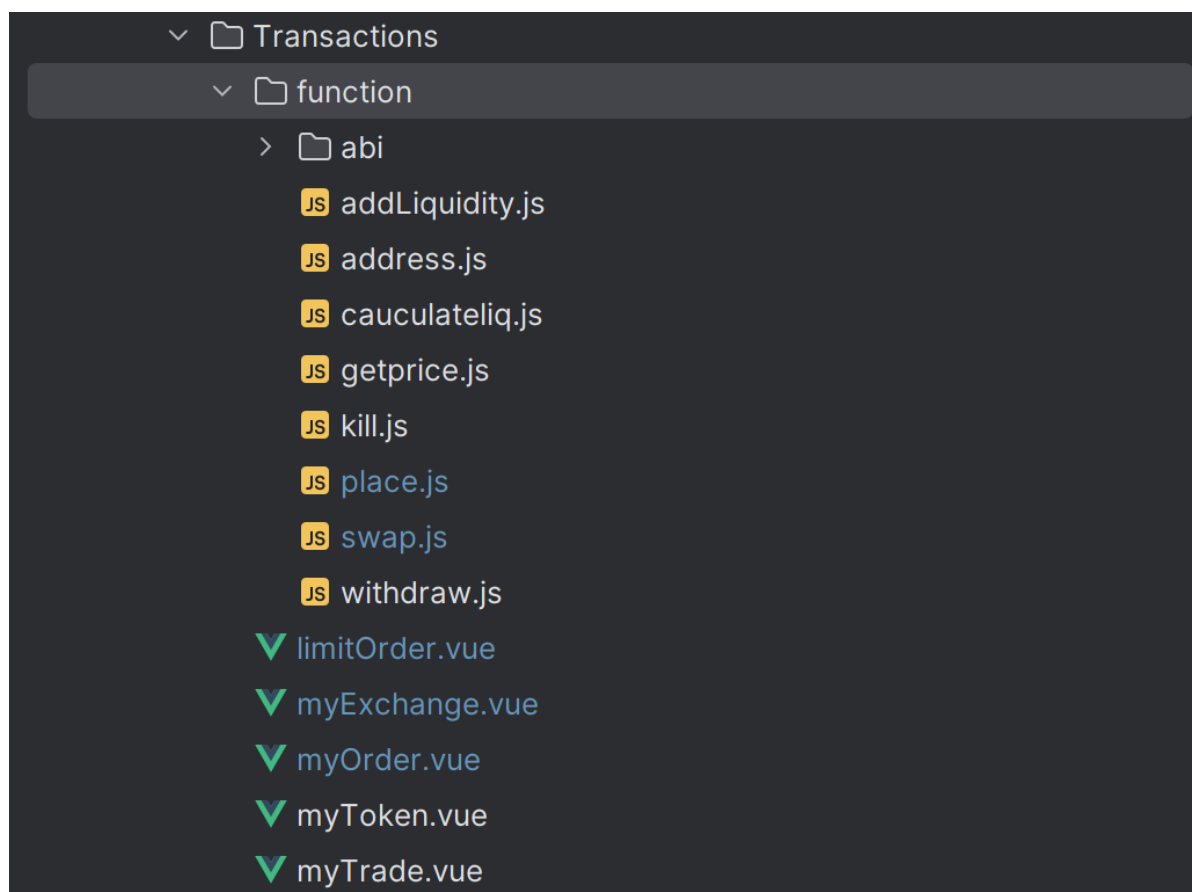
5. `vue.config.js`:

- Vue CLI 的配置文件，用于自定义 Vue CLI 项目的配置，例如修改 Webpack 的配置、更改项目的构建输出路径等。
- 如果使用 Vue CLI 插件，例如 `@vue/cli-plugin-electron-builder`，此文件也可能包含 Electron 的构建和打包设置。

指令介绍:

```
npm install //基本的就这个，下载库的指令，其余的可以查文档解决。
```

vue3与合约交互



这是一个目前基本上完成了界面与功能实现的文件目录。

文件夹下面vue组件为页面级别的组件，而function文件夹中为合约调用的脚本，通过vue的script中的import即可调用调用合约的脚本，利于修改也利于代码的模块化。abi文件夹用于存放合约的abi。

界面风格

每个组可以根据自己的需求进行界面风格的设计和实现，整体上风格较为统一即可。

目前的实现比较简单：以下图为例：



该页面位于src/views/Transactions/limitOrder,一些css样式可以在文件中得到,有需要可以直接使用。

对于操作交互的页面,目前实现均比较简单,总结为以下几点:

- 顶上有一个导航栏
- 白色边框,白色字体,透明背景颜色以适应暗色调的背景图片。
- 对于原生button, input, select,radio等的样式重写,同样设置为白框,白字,透明或接近背景颜色的颜色。
- 边框圆角,按钮的话有非常多的样式按钮,可以自行去网上搜寻css代码并使用自己喜欢的。
- 布局:使用的flex布局,由于操作的main-center部分存在缩放,进行尽可能使用%来控制高度和宽度,减少px的使用。
- 屏幕分辨率适配:这一部分目前项目做的还不完善,无法给到更多的建议。
- 需要使用到分页器的, components中有封装好的myPagination.vue组件。
- 比较复杂的样式,也可以借助UI组件库,例如element-plus, primevue等等,只需要重写样式使得与风格较为统一即可。